

What is Data Preprocessing Funnel?

The **Data Preprocessing Funnel** is a step-by-step process to **clean and prepare raw data** before training a machine learning model.

“Raw data → Clean data → Ready for model”

1. Handling Missing Data

Sometimes data has **missing values (empty cells)**.

Why important:

ML models cannot handle missing values directly.

Example:

Age	Salary
25	30000
30	X

Missing salary needs to be handled

Solutions:

- Fill with mean (average)
- Fill with median
- Remove rows

Encoding Categorical Values

What it means:

Convert **text data into numbers**

Why important:

ML models understand only numbers

Example:

City
Delhi
Mumbai

Convert into:

- Delhi $\rightarrow$ 0
- Mumbai $\rightarrow$ 1

3. Feature Scaling

What it means:

Convert values into the **same scale**

Why important:

Large values can dominate small values

Example:

Age	Salary
25	50000

Salary is much bigger $\rightarrow$ needs scaling

Methods:

- Standardization
- Normalization

4. Splitting Data

What it means:

Divide data into:

- Training set
- Testing set

Why important:

- Training $\rightarrow$ learn
- Testing $\rightarrow$ evaluate

Example:

100 rows:

- 80 $\rightarrow$ Training
- 20 $\rightarrow$ Testing

5. Data Prepared

Meaning:

After all steps:

Data is **clean, numeric, and ready for model**

Full Flow (Funnel)

Raw Data

→ Handle Missing Values

→ Encode Data

→ Scale Features

→ Split Data

→ Ready for ML Model

Complete Example (Student Dataset)

Problem:

Predict student marks

Raw Data:

Hours	City	Marks
2	Delhi	30
3	Mumbai	40
4	X	50

Step 1: Handle Missing Data

Fill missing city:

- Replace with most common value (Delhi)

Step 2: Encode Categorical Data

City
Delhi → 0
Mumbai → 1

New table:

Hours	City	Marks
2	0	30
3	1	40
4	0	50

Step 3: Feature Scaling

Scale Hours (example):

Hours
2 → 0.2
3 → 0.3
4 → 0.4

Step 4: Split Data

Training:

- First 2 rows

Testing:

- Last row

Step 5: Data Ready

Now data is ready for model in
Scikit-learn

Code Example (Scikit-learn)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Example data
X = [[2, 0], [3, 1], [4, 0]]
y = [30, 40, 50]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

How to Clean Data When Data is Raw

Step 1: Take Raw Data in Pandas

```
import pandas as pd

data = {
    "Age": [25, 30, None, 28, 35],
    "Salary": [50000, None, 60000, 55000, None],
    "City": ["Delhi", "Mumbai", "Delhi", None, "Mumbai"]
}

df = pd.DataFrame(data)
```

```
print(df)
```

Raw Data Output:

Age	Salary	City
25	50000	Delhi
30	X	Mumbai
X	60000	Delhi
28	55000	X
35	X	Mumbai

This is **raw data (dirty data)**

Step 2: Understand Mean, Median, Mode

1. Mean (Average)

Formula:

(sum of values) / total values

Example:

$(25 + 30 + 28 + 35) / 4 = \mathbf{29.5}$

2. Median (Middle Value)

Sorted values:

25, 28, 30, 35

Median = $(28 + 30) / 2 = \mathbf{29}$

3. Mode (Most Frequent Value)

Example:

City = Delhi appears most

Mode = **Delhi**

When to use:

Situation	Use
Normal data	Mean
Outliers present	Median
Categorical data	Mode

Step 3: Clean Missing Data

Fill Missing Values

```
# Fill Age with mean
df["Age"].fillna(df["Age"].mean(), inplace=True)

# Fill Salary with median
df["Salary"].fillna(df["Salary"].median(), inplace=True)

# Fill City with mode
df["City"].fillna(df["City"].mode()[0], inplace=True)

print(df)
```

Cleaned Data:

Age	Salary	City
25	50000	Delhi
30	55000	Mumbai
29.5	60000	Delhi
28	55000	Delhi
35	55000	Mumbai

Step 4: Alternative (Remove Missing Data)

```
df.dropna(inplace=True)
```

But:

- Data loss ho sakta hai
- Only use when missing data is small

How Much % Data Should Be Clean?

Practical Rule:

If missing data is:

- < 5% → Ignore or fill easily
- 5% – 20% → Fill using mean/median/mode
- 20% – 40% → Careful handling needed
- > 40% → Consider removing column

Simple Understanding:

Goal:

Keep **maximum useful data**

Avoid too many wrong assumptions

Full Cleaning Workflow

1. Load raw data
2. Check missing values
3. Fill using:
 - Mean (numeric)
 - Median (outliers)
 - Mode (categorical)
4. Verify cleaned data
5. Then move to model

Important Real Insight

In real ML projects:

- 60–70% time = Data Cleaning
- 30–40% = Model building

What is Encoding?

Encoding is the process of converting categorical (text) data into numerical values so that machine learning models can understand it.

Simple Meaning:

“Convert words → numbers”

Example:

City
Delhi
Mumbai

After encoding:

- Delhi → 0
- Mumbai → 1

Why Encoding is Needed?

ML models only understand **numbers**, not text.

Types of Encoding

1. Label Encoding

Label Encoding assigns a **unique number to each category**

Example:

City	Encoded
Delhi	0
Mumbai	1
Chennai	2

Problem:

Model may think:

Mumbai (1) > Delhi (0)

But actually:

There is no order in cities

Code Example:

```
from sklearn.preprocessing import LabelEncoder

cities = ["Delhi", "Mumbai", "Chennai", "Delhi"]

encoder = LabelEncoder()
encoded = encoder.fit_transform(cities)

print(encoded)
```

Output:

```
[0 2 1 0]
```

2. One-Hot Encoding

One-Hot Encoding creates **separate columns for each category**

Example:

City	Delhi	Mumbai	Chennai
Delhi	1	0	0
Mumbai	0	1	0
Chennai	0	0	1

Advantage:

No false ranking

Better for ML models

Code Example:

```
import pandas as pd

df = pd.DataFrame({"City": ["Delhi", "Mumbai", "Chennai"]})
```



```
encoded = pd.get_dummies(df["City"])
print(encoded)
```

Output:

```

      Chennai  Delhi  Mumbai
0           0     1       0
1           0     0       1
2           1     0       0

```

When to Use What?

Encoding Type	Use Case
Label Encoding	When data has order (Low, Medium, High)
One-Hot Encoding	When no order (City, Color, Gender)

Real-Life Example

Problem: Predict Salary

Education
Graduate
Postgraduate
PhD

Label Encoding is OK here (because order exists)

Problem: Predict Customer City

City
Delhi
Mumbai

Use One-Hot Encoding (no order)

Important Tips

Don't use Label Encoding for:

- Cities
- Names
- Categories without order

Use One-Hot Encoding for most cases